

```
=====
Problem: Avg Of Sub Arr Of KSize
Level: Intermediate
Source: Arrays/AvgOfSubArrOfKSize.java
Time complexity: O(n)
Space complexity: O(n)
Your original code:
```

```
import java.util.Scanner;
public class AvgOfSubArrOfKSize{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter n ");
        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }
        System.out.println("Enter k");
        int k = sc.nextInt();

        int sum_of_k = 0;

        for(int i=0;i<k;i++){
            sum_of_k = sum_of_k + arr[i];
        }

        double avg = (double)sum_of_k/k;
        System.out.println(avg);

        for(int i=k;i<n;i++){
            sum_of_k = sum_of_k - arr[i-k] + arr[i];
            avg = (double)(sum_of_k/k);
            System.out.println("Avg of sub array of size k is:" + avg);
        }

    }
}
```

```
=====
Problem: Largest
Level: Intermediate
Source: Arrays/Largest.java
Time complexity: O(n)
Space complexity: O(n)
Your original code:
```

```
import java.util.Scanner;

public class Largest{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int max = arr[0];

        for(int i=1;i<n;i++){
            if(arr[i]>max){
                max = arr[i];
            }
        }
    }
}
```

```

    }
    System.out.println(max);
}
}

```

```

=====
Problem: Max Sum Sub Arr Of Size K
Level: Intermediate
Source: Arrays/MaxSumSubArrOfSizeK.java
Time complexity: O(n)
Space complexity: O(n)
Your original code:

```

```

import java.util.Scanner;

public class MaxSumSubArrOfSizeK{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter n");
        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }
        System.out.println("Enter k");
        int k =sc.nextInt();
        int Sum_of_k = 0;
        for(int i=0;i<k;i++){
            Sum_of_k = Sum_of_k + arr[i];
        }

        int max_sum = Sum_of_k;

        for(int i=k;i<n;i++){
            Sum_of_k = Sum_of_k - arr[i-k]+arr[i];
            if(Sum_of_k > max_sum){
                max_sum = Sum_of_k;
            }
        }
        System.out.println(max_sum);
    }
}

```

```

=====
Problem: Reversing Inside Same Array
Level: Intermediate
Source: Arrays/ReversingInsideSameArray.java
Time complexity: O(n)
Space complexity: O(n)
Your original code:

```

```

import java.util.Scanner;
public class ReversingInsideSameArray{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<arr.length;i++){
            arr[i] = sc.nextInt();
        }

        int start = 0;
        int end = arr.length - 1;

        while(start < end){
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
        }
    }
}

```

```

        start++;
        end--;
    }
    for(int num = 0; num<arr.length; num++){
        System.out.print(arr[num] + " ");
    }
}
}

```

=====

Problem: Second Largest

Level: Intermediate

Source: Arrays/SecondLargest.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.Scanner;
public class SecondLargest{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int largest = Integer.MIN_VALUE;
        int secondlargest = Integer.MIN_VALUE;
        for(int i=0;i<n;i++){
            if(arr[i]>largest){
                secondlargest = largest;
                largest = arr[i];
            }
        }

        for(int i=0;i<n-1;i++){
            if(arr[i]>secondlargest && arr[i] != largest){
                secondlargest = arr[i];
            }
        }
        System.out.println(secondlargest);
    }
}

```

=====

Problem: Smallest

Level: Intermediate

Source: Arrays/Smallest.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.Scanner;

public class Smallest{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int min = arr[0];
        for(int i=1;i<n;i++){
            if(arr[i]<min){
                min = arr[i];
            }
        }
    }
}

```

```

    }
    System.out.println(min);
}
}

```

=====

Problem: Sorted Array

Level: Intermediate

Source: Arrays/SortedArray.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.Scanner;
public class SortedArray{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        for(int i=0;i<n-1;i++){
            boolean isSorted = false;
            for(int j=0;j<n-i-1;j++){
                if(arr[j]>arr[j+1]){
                    isSorted = false;
                    int temp = arr[j];
                    arr[j] = arr[j+1];
                    arr[j+1] = temp;
                }
            }
            if(isSorted){
                break;
            }
        }

        for(int i=0;i<n;i++)
            System.out.print(arr[i] + " ");

    }
}

```

=====

Problem: Sorting An Array

Level: Intermediate

Source: Arrays/SortingAnArray.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.Scanner;
public class SortingAnArray{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        boolean isSorted = true;

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        for(int i=0;i<n-1;i++){
            if(arr[i]>arr[i+1]){
                isSorted = false;
            }
        }

        if(isSorted == true){

```

```

        System.out.print("Array is sorted");
    }
    else
        System.out.print("Array is not sorted");
}
}

```

```

=====
Problem: Sumofarray
Level: Intermediate
Source: Arrays/Sumofarray.java
Time complexity: O(n)
Space complexity: O(n)
Your original code:

```

```

import java.util.*;
public class Sumofarray{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int sum = 0;
        for(int i=0;i<n;i++){
            sum += arr[i];
        }
        System.out.print(sum);
    }
}

```

```

=====
Problem: Time Conversion
Level: Intermediate
Source: Arrays/TimeConversion.java
Time complexity: O(1)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;
class TimeConversion{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        String[] parts = str.split(" ");
        String[] hm = parts[0].split(":");

        int hour = Integer.parseInt(hm[0]);
        int min = Integer.parseInt(hm[1]);

        String period = parts[1];

        if(period.equalsIgnoreCase("AM")){
            if(hour == 12){
                hour = 0;
            }
        } else if(period.equalsIgnoreCase("PM")){
            if(hour != 12){
                hour = hour+12;
            }
        }

        System.out.printf("24-hr format: %02d:%02d\n",hour,min);
        sc.close();
    }
}

```

Problem: palindrome

Level: Intermediate

Source: Arrays/palindrome.java

Time complexity: $O(n)$ Space complexity: $O(n)$

Your original code:

```
import java.util.Scanner;
public class palindrome {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        int original = n;
        int reverse = 0;

        while(n!=0){
            int lastdigit = n%10;
            reverse = reverse*10+lastdigit;
            n = n/10;
        }
        if(original == reverse)
            System.out.print("Palindrom");
        else
            System.out.println("not a palindrome");
    }
}
```

=====

Problem: reverse

Level: Intermediate

Source: Arrays/reverse.java

Time complexity: $O(n)$ Space complexity: $O(n)$

Your original code:

```
import java.util.Scanner;

public class reverse{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        int original = n;
        int reverse = 0;

        while(n != 0){
            int lastdigit = n%10;
            reverse = reverse*10+lastdigit;
            n = n/10;
        }
        System.out.print(reverse);
    }
}
```

=====

Problem: reverse Anarray

Level: Intermediate

Source: Arrays/reverseAnarray.java

Time complexity: $O(n)$ Space complexity: $O(n)$

Your original code:

```
import java.util.Scanner;
public class reverseAnarray{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n =sc.nextInt();
        int[] arr = new int[n];
```

```

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        for(int i=n-1;i>=0;i--){
            System.out.println(arr[i]);
        }
    }
}

```

=====

Problem: Floyd Triangle

Level: Intermediate

Source: Avg problems/FloydTriangle.java

Time complexity: $O(n^2)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```

import java.util.Scanner;

public class FloydTriangle{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int num = 1;
        for(int k=1;k<n;k++){
            for(int j=1;j<=k;j++){
                System.out.print(num + " ");
                num++;
            }

            System.out.println();
        }
    }
}

```

=====

Problem: Min Distance Bet Wrds

Level: Intermediate

Source: Avg problems/MinDistanceBetWrds.java

Time complexity: $O(1)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```

import java.util.Scanner;

public class MinDistanceBetWrds{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        String word1 = sc.nextLine();
        String word2 = sc.nextLine();

        String[] split = str.split(" ");

        int index1 = -1;
        int index2 = -1;
        int min = Integer.MAX_VALUE;

    }
}

```

=====

Problem: Odd Length Diagonal Pattern

Level: Intermediate

Source: Avg problems/OddLengthDiagonalPattern.java

Time complexity: $O(n)$ Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.Scanner;

public class OddLengthDiagonalPattern{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        int n = str.length();

        //rows
        for(int i=0;i<str.length();i++){
            //columns
            for(int j=0;j<str.length();j++){
                //first diagonal
                if(i == j){
                    System.out.print(str.charAt(i));
                }
                //second diagonal
                else if(i+j == n-1){
                    System.out.print(str.charAt(i));
                }
                //else space
                else{
                    System.out.print(" ");
                }
            }

            System.out.println();
        }
    }
}
```

=====

Problem: Reverse Words String

Level: Intermediate

Source: Avg problems/ReverseWordsString.java

Time complexity: $O(n)$ Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.Scanner;

public class ReverseWordsString{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        String[] split = str.split(" ");

        for(int i=split.length-1;i>=0;i--){
            System.out.print(split[i] + " ");
        }
    }
}
```

=====

Problem: Toggle Demo

Level: Intermediate

Source: Avg problems/ToggleDemo.java

Time complexity: $O(1)$ Space complexity: $O(1)$ auxiliary space

Your original code:

```
import javax.swing.*;
```

```

import java.awt.*;
import java.awt.event.*;

class ToggleDemo {
    ToggleDemo() {
        JFrame jf = new JFrame("Toggle Button");

        JLabel lbl = new JLabel("Button is OFF");

        JToggleButton tb =
            new JToggleButton("ON/OFF");

        tb.addItemListener(new ItemListener() {
            public void itemStateChanged(ItemEvent e) {
                if(tb.isSelected())
                    lbl.setText("Button is ON");
                else
                    lbl.setText("Button is OFF");
            }
        });

        jf.setLayout(new FlowLayout());
        jf.add(tb);
        jf.add(lbl);

        jf.setSize(250,150);
        jf.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        jf.setVisible(true);
    }

    public static void main(String[] args) {
        new ToggleDemo();
    }
}

```

```

=====
Problem: Binary Search sorted
Level: Intermediate
Source: Digital/Search_Problems/Binary_Search_sorted.java
Time complexity: O(log n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.*;

public class Binary_Search_sorted{

    public static int Binary_Search(int[] arr , int t){

        int low = 0;
        int high = arr.length-1;

        while(low <= high){
            int mid = low+(high - low)/2;
            if(arr[mid] == t){
                return mid;
            }else if(arr[mid] < t){
                low = mid+1;
            }else{
                high = mid-1;
            }
        }

        return -1;
    }

    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int[] arr = {3, 7, 11, 15, 19, 24, 30};
        int target = 24;
        int result = Binary_Search(arr,target);
        System.out.print(result);
    }
}

```

```
    }
}
```

```
// Time =  $O(\log n)$  ?
// Space =  $O(1)$  for the iterative version. ?
```

```
=====
Problem: First Occurance
Level: Intermediate
Source: Digital/Search_Problems/First_Occurance.java
Time complexity:  $O(\log n)$ 
Space complexity:  $O(1)$  auxiliary space
Your original code:
```

```
import java.util.*;

public class First_Occurance{
    public static int firstOccurance(int[] arr, int t){
        int low = 0;
        int high = arr.length-1;
        int ans = arr.length;

        while(low <= high){
            int mid = low+(high-low)/2;
            if(arr[mid] == t){
                ans = mid;
                high = mid-1;
            } else if(arr[mid] < t){
                low = mid+1;
            }else{
                high = mid-1;
            }
        }
        return ans;
    }
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int[] arr = {1, 2, 4, 4, 4, 7, 9};
        int target = 4;

        int result = firstOccurance(arr,target);
        System.out.print(result);
    }
}
```

```
=====
Problem: Frequency
Level: Intermediate
Source: Digital/Search_Problems/Frequency.java
Time complexity:  $O(\log n)$ 
Space complexity:  $O(1)$  auxiliary space
Your original code:
```

```
import java.util.*;

public class Frequency{
    public static int firstOccurance(int[] arr, int t){
        int low = 0;
        int high = arr.length-1;
        int ans = arr.length;
        while(low <= high){
            int mid = low+(high-low)/2;
            if(arr[mid] == t){
                ans = mid;
                high = mid-1;
            }else if(arr[mid] < t){
                low = mid+1;
            }else{
                high = mid-1;
            }
        }
        return ans;
    }
}
```

```

    }
    public static int lastOccurance(int[] arr,int t){
        int low = 0;
        int high = arr.length-1;
        int ans = arr.length;
        while(low<=high){
            int mid = low+(high-low)/2;
            if(arr[mid] == t){
                ans = mid;
                low = mid+1;
            }else if(arr[mid] > t){
                high = mid-1;
            }else{
                low = mid+1;
            }
        }
        return ans;
    }

    public static void main(String[] args){

        int[] arr = {1, 2, 4, 4, 4, 7, 9};
        int target = 4;
        int frequency = 0;

        int last = lastOccurance(arr,target);
        int first = firstOccurance(arr,target);

        if(first == arr.length){
            frequency = 0;
        }else{
            frequency = last - first + 1;
        }
        System.out.print(frequency);
    }
}

```

=====

Problem: Last Occurance

Level: Intermediate

Source: Digital/Search_Problems/Last_Occurance.java

Time complexity: $O(\log n)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```

import java.util.*;

public class Last_Occurance{
    public static int lastOccurance(int[] arr, int t){
        int low = 0;
        int high = arr.length-1;
        int ans = arr.length;

        while(low <= high){
            int mid = low+(high-low)/2;

            if(arr[mid] == t){
                ans = mid;
                low = mid+1;
            }else if(arr[mid] > t){
                high = mid-1;
            }else{
                low = mid+1;
            }
        }
        return ans;
    }

    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int[] arr = {1, 2, 4, 4, 4, 7, 9};
        int target = 4;
    }
}

```

```

        int result = lastOccurance(arr,target);
        System.out.print(result);
    }
}

```

```

=====
Problem: Linear Search
Level: Intermediate
Source: Digital/Search_Problems/Linear_Search.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.*;

public class Linear_Search{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int[] arr = {5,8,2,10,3,7};
        int target = 3;

        for(int i=0;i<arr.length;i++){
            if(arr[i] == target){
                System.out.print(i);
            }
        }
    }
}

```

```

=====
Problem: Lower Bound
Level: Intermediate
Source: Digital/Search_Problems/Lower_Bound.java
Time complexity: O(log n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.*;

public class Lower_Bound{
    public static int lowerBound(int[] arr, int t){
        int low = 0;
        int high = arr.length-1;
        int ans = arr.length;

        while(low <= high){
            int mid = low+(high-low)/2;
            if(arr[mid] >= t){
                ans = mid;
                high = mid-1;
            }else{
                low = mid+1;
            }
        }
        return ans;
    }

    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int[] arr = {1, 2, 4, 4, 4, 7, 9};
        int target = 4;

        int result = lowerBound(arr,target);
        System.out.print(result);
    }
}

```

```

=====
Problem: Min In Rotated Sorted Array

```

```
import java.util.*;

public class Min_in_RotatedSorted_Array{
    public static int findingMin(int[] arr){
        int low = 0;
        int high = arr.length-1;

        int ans = arr[0];

        while(low<=high){

            if(arr[low] <= arr[high]){
                ans = Math.min(ans, arr[low]);
                break;
            }

            int mid = low+(high-low)/2;

            if(arr[low] <= arr[mid]){
                ans = Math.min(ans, arr[low]);
                low = mid+1;
            }else{
                ans = Math.min(ans, arr[mid]);
                high = mid-1;
            }
        }
        return ans;
    }

    public static void main(String[] args){
        int[] arr = {4,5,6,7,0,1,2};

        int result = findingMin(arr);

        System.out.print(result);
    }
}
```

Problem: Search Insert Position

Level: Intermediate

Source: Digital/Search_Problems/Search_Insert_Position.java

Time complexity: $O(\log n)$ Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.*;

public class Search_Insert_Position{
    public static int Insert_Position(int[] arr, int t){
        int low = 0;
        int high = arr.length-1;

        while(low <= high){
            int mid = low+(high-low)/2;
            if(arr[mid] == t){
                return mid;
            }else if(arr[mid] < t){
                low = mid+1;
            }else{
                high = mid-1;
            }
        }
        return low;
    }

    public static void main(String[] args){
```

```

Scanner sc = new Scanner(System.in);
int[] arr = {1, 3, 5, 6};
int target = 2;

int result = Insert_Position(arr,target);
System.out.print(result);
}
}

```

=====

Problem: Target In Rotated Sorted array

Level: Intermediate

Source: Digital/Search_Problems/Target_In_Rotated_Sorted_array.java

Time complexity: $O(\log n)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```

import java.util.*;

public class Target_In_Rotated_Sorted_array{
    public static int findingTarget(int[] arr, int t){
        int low = 0;
        int high = arr.length-1;
        boolean sorted = false;
        int found = arr.length;

        while(low<=high){
            int mid = low+(high-low)/2;

            if(arr[mid] == t){
                found = mid;
                break;
            }

            if(arr[low] <= arr[mid]){
                sorted = true;
                if(sorted){
                    if(t >= arr[low] && t < arr[mid]){
                        high = mid-1;
                    }else{
                        low = mid+1;
                    }
                }
            } else{
                if(t > arr[mid] && t <= arr[high]){
                    low = mid+1;
                }else{
                    high = mid-1;
                }
            }
        }
        return found;
    }
    public static void main(String[] args){
        int[] arr = {4,5,6,7,0,1,2};
        int target = 8;

        int result = findingTarget(arr,target);

        System.out.print(result);
    }
}

```

=====

Problem: Upper Bound

Level: Intermediate

Source: Digital/Search_Problems/Upper_Bound.java

Time complexity: $O(\log n)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.*;

public class Upper_Bound{
    public static int upperBound(int[] arr, int t){
        int low = 0;
        int high = arr.length-1;
        int ans = arr.length;

        while(low <= high){
            int mid = low+(high-low)/2;
            if(arr[mid]>t){
                ans = mid;
                high = mid-1;
            }else{
                low = mid+1;
            }
        }
        return ans;
    }
    public static void main(String[] args){
        int[] arr = {1, 2, 4, 4, 4, 7, 9};
        int target = 4;

        int result = upperBound(arr,target);
        System.out.print(result);
    }
}
```

=====

Problem: Alternate Uppercase

Level: Intermediate

Source: Strings/AlternateUppercase.java

Time complexity: $O(n)$ Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.Scanner;

public class AlternateUppercase {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        int index = 0;

        for (int i = 0; i < str.length(); i++) {

            char ch = str.charAt(i);

            if (ch == ' ') {
                System.out.print(" ");
                index = 0;
            }
            else
            {
                if(index%2 == 0){
                    System.out.print(Character.toUpperCase(ch));
                }

                else{
                    System.out.print(Character.toLowerCase(ch));
                }
                index++;
            }
        }
    }
}
```

}

=====

Problem: Arithmetic Progression

Level: Intermediate

Source: Strings/ArithmeticProgression.java

Time complexity: $O(1)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.Scanner;
```

```
public class ArithmeticProgression{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int a1 = sc.nextInt();
        int a2 = sc.nextInt();
        int a3 = sc.nextInt();

        int n = sc.nextInt();

        int d = a2-a1;

        int nthNum = a1 + (n-1)*d;

        System.out.print(nthNum);
    }
}
```

=====

Problem: Arrange Alphabets

Level: Intermediate

Source: Strings/ArrangeAlphabets.java

Time complexity: $O(n)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.Scanner;
```

```
public class ArrangeAlphabets{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        char[] ch = str.toCharArray();

        for(int i=0;i<ch.length-1;i++){
            int j=i+1;
            if(ch[i]>ch[j]){
                System.out.print(ch[i]);
            }
            j++;
        }
    }
}
```

=====

Problem: Avg Speed

Level: Intermediate

Source: Strings/AvgSpeed.java

Time complexity: $O(n)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```

import java.util.Scanner;
public class AvgSpeed{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        String[] arr = str.split(" ");

        double distance = 0;
        double time = 0;
        double avg = 0;

        for(int i=0;i<arr.length;i++){
            String[] astric = arr[i].split("@");
            distance = distance + Double.parseDouble(astric[0]);
            time = time + Double.parseDouble(astric[1]);
        }
        avg = distance/time;

        System.out.printf("%.2f kmph",avg);
    }
}

```

```

=====
Problem: Char At Mul Of X
Level: Intermediate
Source: Strings/CharAtMulOfX.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;
public class CharAtMulOfX{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        int num = sc.nextInt();

        for(int i=0;i<str.length();i++){
            char ch = str.charAt(i);

            if((i+1) % num == 0){
                System.out.print(ch);
            }
        }
    }
}

```

```

=====
Problem: Char BFollows A
Level: Intermediate
Source: Strings/CharBFollowsA.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;
public class CharBFollowsA{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        char firstChar = sc.next().charAt(0);
        char secChar = sc.next().charAt(0);

        int count = 0;

```

```

    for(int i=0;i<str.length()-1;i++){
        char ch = str.charAt(i);
        if(str.charAt(i) == firstChar && str.charAt(i+1) == secChar ){
            count++;
        }
    }

    System.out.print(count);
}
}

```

=====

Problem: Character Remove

Level: Intermediate

Source: Strings/CharacterRemove.java

Time complexity: O(1)

Space complexity: O(1) auxiliary space

Your original code:

```

import java.util.Scanner;
public class CharacterRemove{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String str = sc.next();
        int n = str.length();

        if(n > 2){
            String result = str.substring(1, str.length()-3);
            System.out.print(result);
        }
    }
}

```

=====

Problem: Check Leapyear

Level: Intermediate

Source: Strings/CheckLeapyear.java

Time complexity: O(1)

Space complexity: O(1) auxiliary space

Your original code:

```

import java.util.Scanner;
public class CheckLeapyear{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int Year = sc.nextInt();

        if(Year%400 == 0){
            System.out.println("yes");
        }
        else if(Year%100 == 0){
            System.out.println("no");
        }
        else if(Year%4 == 0){
            System.out.println("yes");
        }
        else{
            System.out.println("no");
        }
    }
}

```

=====

Problem: Checking Palindrome Farming

Level: Intermediate

Source: Strings/CheckingPalindromeFarming.java

Time complexity: O(n)

Space complexity: O(n)

Your original code:

```

import java.util.Scanner;
public class CheckingPalindromeFarming{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        int[] freq = new int[256];

        for(int i=0;i<str.length();i++){
            freq[str.charAt(i)]++;
        }

        int oddcount = 0;

        for(int i=0;i<str.length();i++){
            if(freq[str.charAt(i)]%2!=0){
                oddcount++;
            }
        }

        if(oddcount<=1){
            System.out.println("Palindrome is possible");
        }
        else{
            System.out.println("Not Possible");
        }
    }
}

```

```

=====
Problem: Common Factor
Level: Intermediate
Source: Strings/CommonFactor.java
Time complexity: O(n)
Space complexity: O(n)
Your original code:

```

```

import java.util.Scanner;
public class CommonFactor{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int gcd = findGCD(arr[0],arr[1]);

        for(int i=1;i<=gcd;i++){
            if(gcd%i==0){
                System.out.print(i + " ");
            }
        }

        public static int findGCD(int n1 , int n2){
            while(n2!=0){
                int temp = n2;
                n2 = n1%n2;
                n1 = temp;
            }
            return n1;
        }
    }
}

```

}

=====

Problem: Common Part

Level: Intermediate

Source: Strings/CommonPart.java

Time complexity: $O(n)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.Scanner;
public class CommonPart{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String s1 = sc.nextLine();
        String s2 = sc.nextLine();

        String answer = " ";

        for(int len= Math.min(s1.length(),s2.length()); len>=1;len--){
            String part1 = s1.substring(s1.length() - len);
            String part2 = s2.substring(0, len);

            if(part1.equals(part2)){
                answer = part2;
                break;
            }

        }

        System.out.print(answer);
    }
}
```

=====

Problem: Concatenate Two Strings

Level: Intermediate

Source: Strings/ConcatenateTwoStrings.java

Time complexity: $O(n)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.Scanner;

public class ConcatenateTwoStrings{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str1 = sc.nextLine();
        String str2 = sc.nextLine();

        int i =0;

        while(i<str1.length() && i<str2.length()){

            if(str1.charAt(i) < str2.charAt(i)){
                System.out.print(str1+str2);
                break;
            }
            else if(str2.charAt(i) < str1.charAt(i)){
                System.out.print(str2+str1);
                break;
            }

            i++;
        }
    }
}
```

```
=====
Problem: Converting Rup paise
Level: Intermediate
Source: Strings/ConvertingRup_paise.java
Time complexity: O(1)
Space complexity: O(1) auxiliary space
Your original code:
```

```
import java.util.Scanner;

public class ConvertingRup_paise{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        float F = sc.nextFloat();

        float amount = F*100;

        System.out.print(Math.round(amount));
    }
}
```

```
=====
Problem: Correct Operator
Level: Intermediate
Source: Strings/CorrectOperator.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:
```

```
import java.util.Scanner;

public class CorrectOperator{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();

        int A = 0;
        int B = 0;
        int C = 0;
        int num = 0;

        for(int i=0;i<str.length();i++){
            char ch = str.charAt(i);

            if(Character.isDigit(ch)){
                num = num*10+(str.charAt(i) - '0');
            }

            else if(ch == '+' || ch == '-' || ch == '*' || ch == '/'){
                A = num;
                num = 0;
            }
            else if(ch == '='){
                B = num;
                num = 0;
            }

        }

        C = num;
        System.out.println(A + " " + B + " " + C);

        if(A+B == C){
            System.out.println("+");
        }else if(A-B == C){
            System.out.println("-");
        }
    }
}
```

```

        else if(A*B == C){
            System.out.println("");
        }else if(A/B == C){
            System.out.print("/");
        }
    }
}

```

=====

Problem: Count No Of Alphabets

Level: Intermediate

Source: Strings/CountNoOfAlphabets.java

Time complexity: O(n)

Space complexity: O(1) auxiliary space

Your original code:

```

import java.util.Scanner;
public class CountNoOfAlphabets{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();

        int alphabet_Count = 0;

        for(int i=0;i<str.length();i++){
            char ch = str.charAt(i);

            if(Character.isLetter(ch)){
                alphabet_Count++;
                System.out.print(ch + " ");
            }
        }
        System.out.print(alphabet_Count);

    }
}

```

=====

Problem: Count Primes In Range

Level: Intermediate

Source: Strings/CountPrimesInRange.java

Time complexity: O(n)

Space complexity: O(1) auxiliary space

Your original code:

```

import java.util.Scanner;

public class CountPrimesInRange{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n1 = sc.nextInt();
        int n2 = sc.nextInt();

        int totalCount = 0;
        for(int i=n1;i<=n2;i++){

            int count = 0;

            for(int j=1;j<=n2;j++){

                if(i%j == 0){
                    count++;
                }
            }
            if(count == 2){
                totalCount++;
            }
        }
    }
}

```

```

        System.out.print(totalCount);
    }
}

```

```

=====
Problem: Count Words
Level: Intermediate
Source: Strings/CountWords.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;

public class CountWords{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        boolean inWord = false;
        int count = 0;

        for(int i=0;i<str.length();i++){
            char ch = str.charAt(i);
            //checks space
            if(ch != ' '){
                //checks ch is in word or not if it is count++
                //and inWord = true
                if(!inWord){
                    count++;
                    inWord = true;
                }
                //else inWord = false
            } else{
                inWord = false;
            }
        }
        //print the count
        System.out.print(count);
    }
}

```

```

=====
Problem: Counting Pairof Characters
Level: Intermediate
Source: Strings/CountingPairofCharacters.java
Time complexity: O(n)
Space complexity: O(n)
Your original code:

```

```

import java.util.Scanner;

public class CountingPairofCharacters{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s1 = sc.nextLine();
        String s2 = sc.nextLine();

        int[] freq = new int[256];

        for(int i=0;i<s1.length();i++){
            freq[s1.charAt(i)]++;
        }
        int count = 0;
        for(int i=0;i<s2.length();i++){
            if(freq[s2.charAt(i)] > 0) {
                count++;
                freq[s2.charAt(i)]--;
            }
        }
    }
}

```

```

    }
}
System.out.print(count);

}
}

```

=====

Problem: Decodestring

Level: Intermediate

Source: Strings/Decodestring.java

Time complexity: $O(n)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```

import java.util.Scanner;
public class Decodestring{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String str = sc.next();
        for(int i=0;i<str.length();i++){
            char ch = str.charAt(i);
            if(Character.isLetter(ch)){
                int num = 0;
                i++;
                while(i<str.length() && Character.isDigit(str.charAt(i))){
                    num = num*10+(str.charAt(i) - '0');
                    i++;
                }
                for(int j=1;j<=num;j++){
                    System.out.print(ch);
                }
                i--;
            }
        }
    }
}

```

=====

Problem: Difference

Level: Intermediate

Source: Strings/Difference.java

Time complexity: $O(1)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```

import java.util.*;
public class Difference {

    public static void main(String[] args) {
        //Your Code Here
        Scanner sc = new Scanner(System.in);

        int rup1 = sc.nextInt();
        int rup2 = sc.nextInt();
        int w1 = sc.nextInt();
        int w2 = sc.nextInt();

        double land1 = (rup1/5.0)/w1;
        double land2 = (rup2/5.0)/w2;

        double result = land1 - land2;

        if(result < 0){
            result = result*-1;
        }

        System.out.printf("%.2f",result);
    }
}

```

```
}
}
```

```
=====
Problem: Expandalphabets
```

```
Level: Intermediate
```

```
Source: Strings/Expandalphabets.java
```

```
Time complexity: O(n)
```

```
Space complexity: O(1) auxiliary space
```

```
Your original code:
```

```
import java.util.Scanner;

public class Expandalphabets{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        char[] ch = str.toCharArray();

        int count = 0;

        for(int i=0;i<ch.length;i++){
            if(Character.isDigit(ch[i])){
                int digit = ch[i] - '0';
                count = count*10 + digit;
            }

            else if(Character.isLetter(ch[i])){
                for(int j=0;j<count;j++){
                    System.out.print(ch[i]);
                }
                count = 0;
            }
        }
    }
}
```

```
=====
Problem: First Rep From Last
```

```
Level: Intermediate
```

```
Source: Strings/FirstRepFromLast.java
```

```
Time complexity: O(n)
```

```
Space complexity: O(1) auxiliary space
```

```
Your original code:
```

```
import java.util.Scanner;

public class FirstRepFromLast{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        boolean found = false;

        for(int i=str.length()-1;i>=0;i--){
            char ch = str.charAt(i);

            for(int j=i-1;j>=0;j--){
                if(str.charAt(i) == str.charAt(j)){
                    System.out.print(ch);
                    found = true;
                    break;
                }
            }
            if(found == true){
                break;
            }
        }
    }
}
```

```

    }
}
}

```

```

=====
Problem: Fibonacci
Level: Intermediate
Source: Strings/Fibonacci.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;

public class Fibonacci{
    public static void main(String[] args){
        Scanner sc =new Scanner(System.in);
        int n = sc.nextInt();
        long prev1 = 0;
        long prev2 = 1;
        System.out.print(prev1 + " " + prev2 + " ");
        for(int i=0;i<n-2;i++){
            long next = prev1 + prev2;
            System.out.print(next + " ");
            prev1 = prev2;
            prev2 = next;
        }
    }
}

```

```

=====
Problem: First Letter Upper
Level: Intermediate
Source: Strings/FirstLetterUpper.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;
public class FirstLetterUpper{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        String result = "";

        for(int i=0;i<str.length();i++){
            char ch = str.charAt(i);

            if((i==0 && ch != ' ') || (ch != 0 && str.charAt(i-1)==' ')){
                result = result + Character.toUpperCase(ch);
            }
            else
                result += ch;
        }

        System.out.print(result);
    }
}

```

```

=====
Problem: First Non Rep Char
Level: Intermediate
Source: Strings/FirstNonRepChar.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;
public class FirstNonRepChar{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        boolean isUnique = true;

        for(int i=0;i<str.length();i++){
            char ch = str.charAt(i);
            isUnique = true;
            for(int j=0;j<str.length();j++){
                if(i!=j && str.charAt(i) == str.charAt(j)){
                    isUnique = false;
                    break;
                }
            }
            if(isUnique){
                System.out.print(ch);
                break;
            }
        }
    }
}

```

=====

Problem: First Repeating Char
 Level: Intermediate
 Source: Strings/FirstRepeatingChar.java
 Time complexity: $O(n)$
 Space complexity: $O(1)$ auxiliary space
 Your original code:

```

import java.util.Scanner;
public class FirstRepeatingChar{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        boolean found = false;

        for(int i=0;i<str.length();i++){
            char ch = str.charAt(i);
            for(int j=i+1;j<str.length();j++){
                if( str.charAt(i) == str.charAt(j)){
                    found = true;
                    System.out.print(ch);
                    break;
                }
            }

            if(found == true){
                break;
            }
        }
    }
}

```

=====

Problem: HCFofTwoNumbers
 Level: Intermediate
 Source: Strings/HCFofTwoNumbers.java
 Time complexity: $O(n)$
 Space complexity: $O(1)$ auxiliary space
 Your original code:

```

import java.util.Scanner;
public class HCFofTwoNumbers{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();
        int smaller;
        if(a<b){
            smaller = a;
        }
        else
            smaller = b;
        int hcf = 0;
        for(int i=1;i<smaller;i++){
            if(a%i==0 && b%i==0){
                hcf=i;
            }
        }
        System.out.print(hcf);
    }
}

```

=====

Problem: Head Count Of Bird Animal
 Level: Intermediate
 Source: Strings/HeadCountOfBird_Animal.java
 Time complexity: O(1)
 Space complexity: O(1) auxiliary space
 Your original code:

```

import java.util.Scanner;
public class HeadCountOfBird_Animal{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int head = sc.nextInt();
        int legs = sc.nextInt();

        int animals = (legs - (2*head))/2;

        int birds = head - animals;

        System.out.print(birds + " " + animals);

    }
}

```

=====

Problem: Late Comers Meet
 Level: Intermediate
 Source: Strings/LateComersMeet.java
 Time complexity: O(n)
 Space complexity: O(1) auxiliary space
 Your original code:

```

import java.util.Scanner;
public class LateComersMeet{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();

        String[] arr = str.split(" ");
        int count = 0;

        for(int i=0;i<arr.length;i++){
            String[] hours = arr[i].split(":");
            int hr = Integer.parseInt(hours[0]);
            int minute = Integer.parseInt(hours[1]);
            if(hr > 10 || (hr==10 && minute > 0)){
                count++;
            }
        }
    }
}

```

```

    }
}
System.out.println(count);
}
}

```

=====

Problem: Lowest Mileage

Level: Intermediate

Source: Strings/LowestMileage.java

Time complexity: O(n)

Space complexity: O(1) auxiliary space

Your original code:

```

import java.util.Scanner;
public class LowestMileage{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();

        String[] arr = str.split(" ");

        float min_mileage = Integer.MAX_VALUE;

        String car_name = " ";
        for(int i=0;i<arr.length;i++){
            String[] car = arr[i].split("@");
            float mileage = Float.parseFloat(car[1]);
            if(mileage < min_mileage){
                min_mileage = mileage;
                car_name = car[0];
            }
        }
        System.out.println(car_name);
    }
}

```

=====

Problem: Min Size Sub Arr Sum

Level: Intermediate

Source: Strings/MinSizeSubArrSum.java

Time complexity: O(n)

Space complexity: O(n)

Your original code:

```

import java.util.Scanner;

public class MinSizeSubArrSum {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter n");
        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int end = 0; end < n; end++) {
            arr[end] = sc.nextInt();
        }

        int start = 0;

        int sum = 0;
        int end = 0;
        int window_length;
        System.out.println("Enter Targte");
        int target = sc.nextInt();
        int min_length = Integer.MAX_VALUE;

        for (end = 0; end < n; end++) {
            sum = sum + arr[end];

```

```

        while (sum >= target) {

            window_length = end - start + 1;
            if (window_length < min_length) {
                min_length = window_length;
            }

            for (int i = start; i <= end; i++) {
                System.out.print(arr[i] + " ");
            }
            System.out.println(" ");
            sum = sum - arr[start];
            start++;
        }
    }
    System.out.println(min_length);
}
}
}

```

=====

Problem: Missing Alphabet

Level: Intermediate

Source: Strings/MissingAlphabet.java

Time complexity: O(n)

Space complexity: O(n)

Your original code:

```

import java.util.Scanner;

public class MissingAlphabet {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();

        int[] freq = new int[256]; // to store frequency of characters

        // Step 1: Count frequency
        for (int i = 0; i < str.length(); i++) {
            freq[str.charAt(i)]++;
        }

        // Step 2: Find first character with odd count
        for (int i = 0; i < str.length(); i++) {
            if (freq[str.charAt(i)] % 2 != 0) {
                System.out.print(str.charAt(i));
                break;
            }
        }
    }
}

```

=====

Problem: Net Gainor Loss

Level: Intermediate

Source: Strings/NetGainorLoss.java

Time complexity: O(n)

Space complexity: O(1) auxiliary space

Your original code:

```

import java.util.Scanner;

public class NetGainorLoss{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        String[] Roll = str.split(" ");
    }
}

```

```

    int gain = sc.nextInt();
    int loss = sc.nextInt();

    int net = 0;

    for(int i=0;i<Roll.length;i++){
        int ch = Integer.parseInt(Roll[i]);
        if(ch%2 != 0){
            // System.out.println(Roll[i]);
            net = net+gain;
        }
        else
            net = net-loss;
    }

    if(net > 0){
        System.out.print(net);
    }
    else {
        System.out.print(net);
    }
}
}

```

```

=====
Problem: Next Prime
Level: Intermediate
Source: Strings/NextPrime.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;
public class NextPrime{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int next = n+1;

        while(true){
            int count = 0;
            for(int i=1;i<=next;i++){
                if(next%i == 0){
                    System.out.print(next);
                    count++;
                }
            }
            // if(count == 2){
            //     System.out.print(next);
            //     break;
            // }
            next++;
        }
    }
}

```

```

=====
Problem: Odd Length Str Middle Letters
Level: Intermediate
Source: Strings/OddLengthStrMiddleLetters.java
Time complexity: O(1)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;
public class OddLengthStrMiddleLetters{

```

```

public static void main(String[] args){
    Scanner sc = new Scanner(System.in);

    String str = sc.nextLine();

    if(str.length()%2 == 1){
        int mid = str.length()/2;
        System.out.print(str.charAt(mid-1) + "" + str.charAt(mid) + "" + str.charAt(mid+1));
    }
}

```

=====

Problem: One Odd Char XOR

Level: Intermediate

Source: Strings/OneOddCharXOR.java

Time complexity: $O(n)$

Space complexity: $O(1)$ auxiliary space

Your original code:

//we can also check palindrome string by XOR gate in the code because XOR removes pair elements $0 \oplus 0 = 0$
 //this program only works when string has only odd frequency characters. else it won't work

```
import java.util.Scanner;
```

```

public class OneOddCharXOR{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        char result = 0;

        for(int i=0;i<str.length();i++){
            result^=str.charAt(i);
        }
        System.out.println(result);
    }
}

```

=====

Problem: Prime

Level: Intermediate

Source: Strings/Prime.java

Time complexity: $O(n)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.Scanner;
```

```

public class Prime{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        if(n<=1){
            System.out.print("Not Prime");
        }
        boolean isprime = true;

        for(int i=2;i<=Math.sqrt(n);i++){
            if(n%i == 0){
                System.out.print("not prime");
            }
        }
        if(isprime == true){
            System.out.print("Prime");
        }
        else
            System.out.print("Invalid");
    }
}

```

=====

Problem: Primetest

Level: Intermediate

Source: Strings/Primetest.java

Time complexity: $O(n)$ Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.Scanner;

public class Primetest{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        boolean isprime = true;

        if(n<=1){
            isprime = false;
        }else{
            for(int i=2;i<=n/2;i++){
                if(n%i==0){
                    isprime = false;
                    break;
                }
            }
            if(isprime){
                System.out.print("Prime");
            }
            else{
                System.out.print("Not Prime");
            }
        }
    }
}
```

=====

Problem: Reverse String Until Underscore

Level: Intermediate

Source: Strings/ReverseStringUntilUnderscore.java

Time complexity: $O(n)$ Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.Scanner;

public class ReverseStringUntilUnderscore{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        char[] ch = str.toCharArray();
        int count;
        boolean found = false;

        for(int i=0;i<ch.length;i++){

            if(ch[i] == '_'){
                found = true;
                count = i;
                System.out.println(count);
                for(int j=count-1;j>=0;j--){
                    System.out.print(ch[j]);
                }
                System.out.print(ch[i]);
                for(int k=i+1;k<ch.length;k++){
                    System.out.print(ch[k]);
                }
                break;
            }
        }

        if(!found){
            for(int h=ch.length-1;h>=0;h--){
```

```

        System.out.print(ch[h]);
    }
}
}

```

```

=====
Problem: Reversesign
Level: Intermediate
Source: Strings/Reversesign.java
Time complexity: O(1)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;

public class Reversesign{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int num = sc.nextInt();

        if(num == 0){
            System.out.print(num);
        }
        else if(num < 0){
            System.out.print(num*(-1));
        }

        else if(num > 0){
            System.out.print(num*(-1));
        }
    }
}

```

```

=====
Problem: SCC
Level: Intermediate
Source: Strings/SCC.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.*;
public class SCC {

    public static void main(String[] args) {
        //Your Code Here
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine().trim();

        int i=0;

        int sign = 1;
        if(i<str.length() && str.charAt(i) == '-'){
            sign = -1;
            i++;
        }

        int result = 0;
        while(i < str.length() && Character.isDigit(str.charAt(i))){
            result = result*10+(str.charAt(i) - '0');
            i++;
        }

        result = result*sign;

        while(i < str.length()){
            char op = str.charAt(i);

```

```

        i++;

        sign = 1;
        if(i<str.length() && str.charAt(i) == '-'){
            sign = -1;
            i++;
        }

        int num = 0;
        while(i < str.length() && Character.isDigit(str.charAt(i))){
            num = num*10+(str.charAt(i) - '0');
            i++;
        }

        num = num*sign;

        if(op == 'A' || op == 'a'){
            result = result + num;
        }else if(op == 'S' || op == 's'){
            result = result - num;
        }
        else if(op == 'M' || op == 'm'){
            result = result*num;
        }else if(op == 'D' || op == 'd'){
            result = result/num;
        }
    }
    System.out.print(result);

}
}

```

=====

Problem: Simple Calc Command

Level: Intermediate

Source: Strings/SimpleCalcCommand.java

Time complexity: O(n)

Space complexity: O(1) auxiliary space

Your original code:

```

import java.util.Scanner;

public class SimpleCalcCommand{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        int i = 0;

        // ? get first number
        int result = 0;
        while(i < str.length() && Character.isDigit(str.charAt(i))){
            result = result * 10 + (str.charAt(i) - '0');
            i++;
        }

        // ? process remaining
        while(i < str.length()){
            char op = str.charAt(i);
            i++;

            int num = 0;

            while(i < str.length() && Character.isDigit(str.charAt(i))){
                num = num * 10 + (str.charAt(i) - '0');
                i++;
            }

            if(op == 'a'){
                result = result + num;
            }
        }
    }
}

```

```

        else if(op == 's'){
            result = result - num;
        }
        else if(op == 'm'){
            result = result * num;
        }
        else if(op == 'd'){
            result = result / num;
        }
    }

    System.out.println(result);
}
}

```

```

=====
Problem: Sumof Int
Level: Intermediate
Source: Strings/SumofInt.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;

public class SumofInt{
    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();

        String[] arr = str.split(" ");

        int sum = 0;

        for(int i=0;i<arr.length;i++){
            int num = Integer.parseInt(arr[i]);
            sum = sum+num;
        }

        System.out.print(sum);
    }
}

```

```

=====
Problem: Tengthunitsum
Level: Intermediate
Source: Strings/Tengthunitsum.java
Time complexity: O(1)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;
public class Tengthunitsum{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int sum = 0;
        sum = n%100+(n/100)%100;
        System.out.print(sum);
    }
}

```

```

=====
Problem: Top Scoring Batsman
Level: Intermediate
Source: Strings/TopScoringBatsman.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:

```

//it is also same as the previous problem but in this code we are accepting and splitting and also assigning values in loop itself by running loop from 0 to n
 import java.util.Scanner;

```
public class TopScoringBatsman{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        sc.nextLine();

        int top_scoring_batsman = Integer.MIN_VALUE;
        String batsman = "";

        for(int i=0;i<n;i++){
            String str = sc.nextLine();
            String[] coma = str.split(",");
            int scorer = Integer.parseInt(coma[1]);
            if(scorer > top_scoring_batsman){
                top_scoring_batsman = scorer;
                batsman = coma[0];
            }
        }
        System.out.print(batsman);
    }
}
```

=====

Problem: Top Scoring Student

Level: Intermediate

Source: Strings/TopScoringStudent.java

Time complexity: O(n)

Space complexity: O(1) auxiliary space

Your original code:

```
import java.util.Scanner;
public class TopScoringStudent{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        sc.nextLine();

        int top_scorer = Integer.MIN_VALUE;
        String Student = "";

        for(int i=0;i<n;i++){
            String str = sc.nextLine();
            String[] arr = str.split(":");
            int score = (Integer.parseInt(arr[1])+Integer.parseInt(arr[2])+Integer.parseInt(arr[3]));
            if(score > top_scorer){
                top_scorer = score;
                Student = arr[0];
            }
        }
        System.out.print(Student);
    }
}
```

=====

Problem: Top Scoring Batsman

Level: Intermediate

Source: Strings/Top_ScoringBatsman.java

Time complexity: O(n)

Space complexity: O(1) auxiliary space

Your original code:

```

import java.util.Scanner;

public class Top_ScoringBatsman{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        String[] arr = str.split(" ");
        int top_scorer = Integer.MIN_VALUE;
        String Batsman = " ";

        for(int i=0;i<arr.length;i++){
            String[] coma = arr[i].split(",");
            int scorer = Integer.parseInt(coma[1]);
            if(scorer > top_scorer){
                Batsman = coma[0];
                top_scorer = scorer;
            }
        }
        System.out.println(Batsman);
    }
}

```

```

=====
Problem: Uppercase Count
Level: Intermediate
Source: Strings/UppercaseCount.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;
public class UppercaseCount{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.next();
        int n = str.length();
        int count = 0;

        for(int i=0;i<n;i++){
            char ch = str.charAt(i);
            if(Character.isUpperCase(ch)){
                count++;
            }
        }
        System.out.print(count);
    }
}

```

```

=====
Problem: interlace
Level: Intermediate
Source: Strings/interlace.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;
public class interlace{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int A = sc.nextInt();
        int B = sc.nextInt();
    }
}

```

```

int odd = (A % 2 == 1)? A : A+1;
int even = (B % 2 == 0)? B : B-1;

int oddcount = (B+1)/2 - A/2;
int evencount = B/2 - (A-1)/2;

boolean startOdd = (oddcount >= evencount);

while(odd <= B || even >= A){

    if(startOdd){
        if(odd <= B){
            System.out.print(odd + " ");
            odd += 2;

        }
        if(even >= A){
            System.out.print(even + " ");
            even -= 2;
        }
    }
    else{
        if(even >= A){
            System.out.print(even + " ");
            even -= 2;
        }

        if(odd <= B){
            System.out.print(odd + " ");
            odd += 2;
        }
    }
}
}
}

```

=====

Problem: reverse String

Level: Intermediate

Source: Strings/reverseString.java

Time complexity: O(n)

Space complexity: O(1) auxiliary space

Your original code:

```

import java.util.Scanner;
public class reverseString{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String str = sc.next();
        int n = str.length();

        char[] arr = new char[n];
        for(int i=0;i<n;i++){
            arr[i] = str.charAt(i);
        }

        int j = 0;
        char[] reverse = new char[n];

        for(int i=n-1;i>=0;i--){
            reverse[j] = arr[i];
            j++;
        }
        for(j=0;j<=n-1;j++)
            System.out.print(reverse[j]);
    }
}

```

=====

Problem: Minimum Distance Bet Words

Level: Intermediate

Source: skillrack/MinimumDistanceBetWords.java

Time complexity: $O(n)$ Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.Scanner;

class MinimumDistanceBetWords {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        String w1 = sc.nextLine();
        String w2 = sc.nextLine();

        String[] arr = str.split(" ");

        int min = Integer.MAX_VALUE;

        if (w1.equals(w2)) {
            int prev = -1;

            for (int i = 0; i < arr.length; i++) {
                if (arr[i].equals(w1)) {
                    if (prev != -1) {
                        min = Math.min(min, i - prev);
                    }

                    prev = i;
                }
            }
        } else {
            int pos1 = -1;
            int pos2 = -1;

            for (int i = 0; i < arr.length; i++) {
                if (arr[i].equals(w1)) {
                    pos1 = i;
                }

                if (arr[i].equals(w2)) {
                    pos2 = i;
                }

                if (pos1 != -1 && pos2 != -1) {
                    min = Math.min(min, Math.abs(pos1 - pos2));
                }
            }

            System.out.println(min);
        }
    }
}
```

 Problem: Tower Line Sight Issue

Level: Intermediate

Source: skillrack/TowerLineSightIssue.java

Time complexity: $O(1)$ Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.Scanner;
```

```

class TowerLineSightIssue{
    static boolean onSegment(int x1, int y1, int x2, int y2, int x, int y){
        if((x2-x1)*(y-y1) != (y2-y1)*(x-x1)){
            return false;
        }

        if(x>=Math.min(x1,x2) && x<=Math.max(x1,x2) && y>=Math.min(y1,y2) && y<=Math.max(y1,y2)){
            return true;
        }

        return false;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int ax = sc.nextInt();
        int ay = sc.nextInt();
        int bx = sc.nextInt();
        int by = sc.nextInt();
        int cx = sc.nextInt();
        int cy = sc.nextInt();
        int dx = sc.nextInt();
        int dy = sc.nextInt();

        if(onSegment(ax,ay,cx,cy,bx,by)){
            System.out.println("yes");
        }
        else if(onSegment(ax,ay,cx,cy,dx,dy)){
            System.out.println("yes");
        }
        else if(onSegment(bx,by,dx,dy,ax,ay)){
            System.out.println("yes");
        }
        else if(onSegment(bx,by,dx,dy,cx,cy)){
            System.out.println("yes");
        }
        else
            System.out.println("no");

    }
}

```

```

=====
Problem: Anagram
Level: Intermediate
Source: tcs/Anagram.java
Time complexity: O(n log n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.*;
class Anagram{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String s1 = sc.nextLine();
        String s2 = sc.nextLine();
        char[] a = s1.toCharArray();
        char[] b = s2.toCharArray();
        Arrays.sort(a);
        Arrays.sort(b);
        System.out.println(Arrays.equals(a,b) ? "Anagram" : "Not Anagram");
    }
}

```

```

=====
Problem: Binary Search
Level: Intermediate
Source: tcs/BinarySearch.java
Time complexity: O(log n)
Space complexity: O(n)
Your original code:

```

```

import java.util.*;

```

```

class BinarySearch{
public static void main(String[] args){
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] arr = new int[n];
for(int i = 0; i < n; i++)
arr[i] = sc.nextInt();
int key = sc.nextInt();
int low = 0, high = n - 1;
boolean found = false;
while(low <= high){
int mid = (low + high) / 2;
if(arr[mid] == key){
found = true;
break;
}
else if(arr[mid] < key)
low = mid + 1;
else
high = mid - 1;
}
System.out.println(found ? "Found" : "Not Found");
}
}

```

```

=====
Problem: Bubble Sort
Level: Intermediate
Source: tcs/BubbleSort.java
Time complexity: O(n?)
Space complexity: O(n)
Your original code:

```

```

import java.util.*;
class BubbleSort{
public static void main(String[] args){
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] arr = new int[n];
for(int i = 0; i < n; i++)
arr[i] = sc.nextInt();
for(int i = 0; i < n-1; i++){
for(int j = 0; j < n-i-1; j++){
if(arr[j] > arr[j+1]){
int temp = arr[j];
arr[j] = arr[j+1];
arr[j+1] = temp;
}
}
}
for(int num : arr)
System.out.print(num + " ");
}
}

```

```

=====
Problem: Linear Search
Level: Intermediate
Source: tcs/LinearSearch.java
Time complexity: O(n)
Space complexity: O(n)
Your original code:

```

```

import java.util.*;
class LinearSearch{
public static void main(String[] args){
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] arr = new int[n];
for(int i = 0; i < n; i++)
arr[i] = sc.nextInt();
int key = sc.nextInt();

```

```
boolean found = false;
for(int num : arr){
    if(num == key){
        found = true;
        break;
    }
}
System.out.println(found ? "Found" : "Not Found");
}
```

=====

Problem: Remove Duplicates
 Level: Intermediate
 Source: tcs/RemoveDuplicates.java
 Time complexity: O(n)
 Space complexity: O(1) auxiliary space
 Your original code:

```
import java.util.*;
class RemoveDuplicates{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();
        String result = "";
        for(char ch : str.toCharArray()){
            if(result.indexOf(ch) == -1)
                result += ch;
        }
        System.out.println(result);
    }
}
```

=====

Problem: Second Largest
 Level: Intermediate
 Source: tcs/SecondLargest.java
 Time complexity: O(n)
 Space complexity: O(n)
 Your original code:

```
import java.util.*;
class SecondLargest{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i = 0; i < n; i++)
            arr[i] = sc.nextInt();
        int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
        for(int num : arr){
            if(num > first){
                second = first;
                first = num;
            } else if(num > second && num != first){
                second = num;
            }
        }
        System.out.println(second);
    }
}
```

=====

Problem: Selection Sort
 Level: Intermediate
 Source: tcs/SelectionSort.java
 Time complexity: O(n)
 Space complexity: O(n)
 Your original code:

```
import java.util.*;
```

```
class SelectionSort {  
  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int n = sc.nextInt();  
        int[] arr = new int[n];  
        for (int i = 0; i < n; i++) {  
            arr[i] = sc.nextInt();  
        }  
        for (int i = 0; i < n - 1; i++) {  
            int minIndex = i;  
            for (int j = i + 1; j < n; j++) {  
                if (arr[j] < arr[minIndex]) {  
                    minIndex = j;  
                }  
            }  
            int temp = arr[i];  
            arr[i] = arr[minIndex];  
            arr[minIndex] = temp;  
        }  
        for (int num : arr) {  
            System.out.print(num + " ");  
        }  
    }  
}
```